



UTEC

Universidad Tecnológica

Devolución prueba final

Preguntas teórico-prácticas · Java

Preguntas 1 a 7 · Sin CodeRunner

Área Programación

Preguntas de opción múltiple

Cada pregunta se muestra primero. La diapositiva siguiente contiene la respuesta correcta y una explicación técnica breve.

7 preguntas · Excepciones · if · procedimientos · interfaces · operadores

- Se revisa la lógica de ejecución paso a paso.
- Se diferencia sintaxis, compilación, ejecución y resultado en consola.
- El objetivo es reforzar conceptos para el examen del 17/07.

Excepciones personalizadas en Java

¿Cuál de las siguientes opciones describe mejor una característica de las excepciones personalizadas en Java?

- a. Deben estar declaradas en la firma del método que las lanza.
- b. Pueden ser lanzadas solo dentro de un bloque try-catch.
- c. Siempre deben extender la clase Exception.
- d. Permiten crear tipos específicos de errores y controlarlos de manera personalizada.

Respuesta correcta y explicación

Correcta: d

Una excepción personalizada representa un error específico del dominio del programa.

- Permite modelar situaciones de error propias de la aplicación.
- Puede extender Exception si se desea una excepción verificada.
- También puede extender RuntimeException si se desea una excepción no verificada.
- No necesariamente debe declararse siempre en la firma: depende de su jerarquía.

```
class SaldoInsuficienteException
    extends Exception { ... }

throw new SaldoInsuficienteException();
```

Secuencia de ejecución con try-catch-finally

Si en el Bloque 2 se produce una excepción del tipo `NullPointerException`, ¿cuál sería la secuencia de ejecución del programa?

```
// Bloque1
try {
    // Bloque2
} catch (ArrayIndexOutOfBoundsException ex) {
    // Bloque3
} catch (Exception e) {
    // Bloque4
} finally {
    // Bloque5
}
// Bloque6
```

- a. Bloque 1 -> Bloque 2 -> Bloque 3 -> Bloque 5 -> Bloque 6
- b. Bloque 1 -> Bloque 2 -> se detiene el programa y lanza una excepción.
- c. Bloque 1 -> Bloque 2 -> Bloque 4 -> Bloque 5 -> Bloque 6
- d. Bloque 1 -> Bloque 2 -> Bloque 5 -> Bloque 6

Respuesta correcta y explicación

Correcta: c

NullPointerException es capturada por catch (Exception e).

- Primero se ejecuta el código anterior al try y luego el Bloque 2.
- El catch de `ArrayIndexOutOfBoundsException` no coincide con `NullPointerException`.
- El catch genérico `Exception` sí captura la excepción.
- El bloque finally siempre se ejecuta antes de continuar con el Bloque 6.

```
Bloque1  
Bloque2  
catch (Exception e) → Bloque4  
finally → Bloque5  
Bloque6
```

Asignación dentro de una condición if

Al ejecutar este programa, ¿qué se muestra en consola?

```
public class EjercicioIf {  
    public static void main(String[] args) {  
  
        int a = 8;  
        int b = 4;  
        int suma = a + b;  
        int resultado = suma * 3;  
        resultado = resultado - 0;  
  
        if (resultado = 24) {  
            System.out.println("Resultado correcto");  
        } else {  
            System.out.println("Resultado distinto");  
        }  
    }  
}
```

- a. Resultado correcto
- b. Al ejecutar el programa se genera una excepción.
- c. El programa no compila.
- d. Resultado distinto

Respuesta correcta y explicación

Correcta: c

En Java, la condición de un if debe ser booleana.

- La expresión `resultado = 24` es una asignación, no una comparación.
- Esa asignación devuelve un `int`, pero el `if` necesita un valor booleano.
- Por eso el error ocurre en compilación, antes de ejecutar el programa.
- La comparación correcta sería `resultado == 24`.

```
resultado = 24 // asigna
resultado == 24 // compara

if (resultado == 24) { ... }
```

Procedimiento con LinkedList y filtro por edad

¿Qué hace el siguiente procedimiento?

```
public static void buscoPersonas(LinkedList<Persona> cli, int edad) {  
    for (Persona p : cli)  
        if (p.getEdad() < edad)  
            System.out.println(p);  
}
```

Seleccionar la opción correcta:

- a. Imprime en consola las personas de la lista que son menores a la edad recibida por argumento.
- b. No funciona, porque está mal el for.
- c. No funciona, porque está mal el if.
- d. Imprime en consola las personas de la lista que son menores a la edad recibida por parámetro.

Respuesta correcta y explicación

Correcta: d

El procedimiento recorre la lista y filtra por edad.

- El parámetro edad se recibe en la firma del método.
- El for-each recorre cada objeto Persona de la lista cli.
- El if evalúa si la edad de la persona es menor que el valor recibido.
- Si la condición es verdadera, imprime el objeto Persona en consola.

```
for (Persona p : cli)
    p.getEdad() < edad
    → imprime p
```

Interfaces en Java

¿Cuáles de las siguientes afirmaciones **NO** son verdaderas?

- 1 - Una interfaz en Java puede contener métodos abstractos y métodos con implementación por defecto (default).
- 2 - Una interfaz puede tener atributos de instancia que cambian su valor durante la ejecución.
- 3 - Las interfaces pueden extender de otras interfaces.
- 4 - Una interfaz puede crear instancias usando el operador new.
- 5 - Una interfaz no puede implementar métodos del tipo default.

a. 3 - 4 - 5

b. 2 - 4 - 5

c. 1 - 3

d. 2 - 4

Respuesta correcta y explicación

Correcta: b

Las afirmaciones falsas son 2, 4 y 5.

- Una interfaz no define atributos de instancia modificables; sus campos son constantes `public static final`.
- No se puede instanciar directamente una interfaz con `new`.
- Desde Java 8, una interfaz puede tener métodos default con implementación.
- Sí es verdadero que una interfaz puede extender otra interfaz.

2 → falso

4 → falso

5 → falso

1 y 3 → verdaderas

Asignación dentro de una condición if

Al ejecutar este programa, ¿qué se muestra en consola?

```
public class EjercicioIf {  
    public static void main(String[] args) {  
  
        int a = 8;  
        int b = 4;  
        int suma = a + b;  
        int resultado = suma * 3;  
        resultado = resultado - 0;  
  
        if (resultado = 24) {  
            System.out.println("Resultado correcto");  
        } else {  
            System.out.println("Resultado distinto");  
        }  
    }  
}
```

- a. Al ejecutar el programa se genera una excepción.
- b. Resultado correcto
- c. Resultado distinto
- d. El programa no compila.

Respuesta correcta y explicación

Correcta: d

El problema no es de ejecución: es de compilación.

- resultado = 24 intenta asignar un valor entero dentro del if.
- Java no acepta enteros como condición lógica.
- Para comparar igualdad debe utilizarse el operador ==.
- Por eso no se imprime ninguna de las dos frases.

```
if (resultado = 24) // incorrecto  
if (resultado == 24) // correcto
```

Operadores, ciclos y evaluación booleana

Dadas las afirmaciones indicadas, ¿cuáles son verdaderas?

```
public class Operadores {  
    public static void main(String[] args) {  
        int a=10, b=-20, c=50;  
        a--;  
        a--;  
        b++;  
        for (int i = 1; i < 5 ; i++) {  
            b++;  
        }  
        boolean oper= a+b > 10 || c+b < 10;  
        int suma=a+b+c;  
        System.out.println(oper);  
        System.out.println(suma);  
    }  
}
```

a. 1 y 3

b. 2 y 4

c. 1 y 4

d. 2 y 3

Dada las siguientes afirmaciones :

- 1) El valor de la variable suma la terminar el programa es 46.
- 2) El valor que se imprime de la variable suma por consola es 43.
- 3) El valor de la variable oper al terminar el programa es true.
- 4) El valor de la variable oper que se imprime por consola al terminar el programa es false.

¿ Cuales de las siguientes afirmaciones son verdaderas ?

Respuesta correcta y explicación

Correcta: b

Las afirmaciones verdaderas son 2 y 4.

- a inicia en 10 y se decrementa dos veces: queda en 8.
- b inicia en -20, se incrementa una vez y luego cuatro veces dentro del for: queda en -15.
- $\text{suma} = 8 + (-15) + 50 = 43$.
- `oper` evalúa `false || false`, por lo tanto queda en `false`.

```
a = 10 → 9 → 8  
b = -20 → -19 → -15  
suma = 8 - 15 + 50 = 43  
oper = false || false = false
```

Switch, break y caída entre casos

¿Qué se imprime por consola al ejecutar el programa?

```
public class ProbarSwitch {  
    public static void main(String[] args) {  
        for (int i = 4; i > 0; i--) {  
            switch (i - 1) {  
                case 0:  
                    System.out.print("A-");  
                    break;  
                case 1:  
                    System.out.print("B-");  
                case 2:  
                    System.out.print("C-");  
                    break;  
                default:  
                    System.out.print("D-");  
            }  
        }  
    }  
}
```

a. D-C-B-C-A-

b. A-B-C-D-

c. D-B-A-C

d. D-C-B-C-A-B-C-

Respuesta correcta y explicación

Correcta: a. D-C-B-C-A-

El switch evalúa $i - 1$ y algunos casos terminan con break.

- El ciclo recorre $i = 4, 3, 2$ y 1 .
- La expresión del switch es $i - 1$, por lo tanto se evalúan $3, 2, 1$ y 0 .
- Para 3 se ejecuta default e imprime D-.
- Para 1 imprime B- y, al no tener break, continúa en case 2 e imprime C-.
- La salida final es D-C-B-C-A-.

```
i=4 → switch(3) → D-  
i=3 → switch(2) → C-  
i=2 → switch(1) → B- y C-  
i=1 → switch(0) → A-
```

Salida: D-C-B-C-A-

Herencia e instanceof

¿Qué se imprime por consola al ejecutar este programa?

```
public class Herencia {  
    public static void main(String[] args) {  
        Vehiculo vehiculo = new Vehiculo();  
        Vehiculo auto = new Auto();  
        Vehiculo moto = new Moto();  
  
        System.out.print((vehiculo instanceof Vehiculo) + " - ");  
        System.out.print((vehiculo instanceof Auto) + " - ");  
        System.out.print((vehiculo instanceof Moto) + " - ");  
        System.out.print((auto instanceof Vehiculo) + " - ");  
        System.out.print((auto instanceof Moto) + " - ");  
        System.out.print((moto instanceof Vehiculo) + " - ");  
    }  
}
```

a. true - false - false - true - false - true -

b. true - false - true - false - false - true -

c. false - false - true - true - true - true -

d. true - false - false - true - true - false -

Respuesta correcta y explicación

Correcta: a. true - false -
false - true - false - true -

instanceof verifica el tipo real del objeto y su jerarquía.

- Un objeto Vehiculo es instancia de Vehiculo, pero no de Auto ni de Moto.
- Un objeto Auto también es Vehiculo, porque Auto hereda de Vehiculo.
- Un objeto Auto no es Moto: son subclases distintas.
- Un objeto Moto también es Vehiculo.
- Por eso la secuencia correcta es true - false - false - true - false - true -.

```
vehiculo instanceof Vehiculo → true  
vehiculo instanceof Auto    → false  
vehiculo instanceof Moto     → false  
auto instanceof Vehiculo     → true  
auto instanceof Moto         → false  
moto instanceof Vehiculo     → true
```

Sobrecarga de métodos y selección por tipo

¿Qué se imprime en consola?

```
public class TestSumar {  
    public static void main(String[] args) {  
        Sumar sumar = new Sumar();  
        sumar.sumar(10, 10);  
    }  
}  
  
public class Sumar {  
    public void sumar(int a, int b) {  
        System.out.println("Entre a int " + (a+b));  
    }  
    public void sumar(short a, short b) {  
        System.out.println("Entre a short " + (a+b));  
    }  
    public void sumar(byte a, byte b) {  
        System.out.println("Entre a byte " + (a+b));  
    }  
}
```

a. Entre a int 20

b. Entre a int 1010

c. Entre a short 20

d. Entre a byte 20

Respuesta correcta y explicación

Correcta: a. Entre a int 20

Los literales enteros 10 y 10 son de tipo int.

- Java selecciona el método sobrecargado según la firma más adecuada.
- Los valores 10 y 10, escritos sin conversión explícita, son int.
- Por eso se invoca sumar(int a, int b).
- La suma numérica 10 + 10 produce 20.
- La salida es Entre a int 20.

```
sumar.sumar(10, 10)  
10 y 10 → int
```

```
Método invocado:  
sumar(int a, int b)
```

```
Salida: Entre a int 20
```

LinkedList, Collections.sort y reverse

¿Qué se imprime por consola al ejecutar este programa?

```
public class PruebaLista {  
    public static void main(String[] args) {  
        List<String> nombres = new LinkedList<>();  
        nombres.add("Gonzalo");  
        nombres.add("Ariel");  
        nombres.set(0, "Blanca");  
        nombres.add("Hector");  
        nombres.add("Pablo");  
        nombres.add("Daniel");  
        nombres.remove(2);  
        nombres.set(2, "Juan");  
        Collections.sort(nombres);  
        Collections.reverse(nombres);  
        nombres.add("Ernesto");  
        nombres.add(3, "Nair");  
        System.out.println(nombres);  
    }  
}
```

a. [Juan, Daniel, Blanca, Nair, Ariel, Ernesto]

b. [Blanca, Nair, Ariel, Ernesto, Juan, Hector]

c. [Daniel, Gonzalo, Blanca, Juan, Nair, Ernesto]

d. [Juan, Blanca, Nair, Ariel, Ernesto, Hector, Gonzalo]

Respuesta correcta y explicación

Correcta: a. [Juan, Daniel, Blanca, Nair, Ariel, Ernesto]

La lista se modifica paso a paso por índice y luego se ordena.

- `set(0,"Blanca")` reemplaza Gonzalo por Blanca.
- `remove(2)` elimina Hector y `set(2,"Juan")` reemplaza Pablo por Juan.
- `sort` ordena alfabéticamente: [Ariel, Blanca, Daniel, Juan].
- `reverse` invierte el orden: [Juan, Daniel, Blanca, Ariel].
- Luego se agrega Ernesto al final y Nair en la posición 3.

```
Luego de sort:  
[Ariel, Blanca, Daniel, Juan]
```

```
Luego de reverse:  
[Juan, Daniel, Blanca, Ariel]
```

```
add("Ernesto")  
add(3,"Nair")
```

```
Resultado:  
[Juan, Daniel, Blanca, Nair, Ariel, Ernesto]
```

Stack: push y pop

¿Qué se imprime en consola?

```
public class Principal {  
    public static void main(String[] args) {  
        Stack<String> nombres = new Stack<>();  
        nombres.push("Antonio");  
        nombres.push("Carmen");  
        nombres.push("Mariana");  
        nombres.push("Dante");  
        nombres.pop();  
        nombres.pop();  
        nombres.push("Oscar");  
        System.out.println(nombres);  
    }  
}
```

a. [Antonio, Carmen, Mariana, Dante, Oscar]

b. [Antonio, Carmen, Oscar]

c. [Dante, Antonio, Oscar]

d. [Mariana, Dante, Oscar]

Respuesta correcta y explicación

Correcta: b. [Antonio,
Carmen, Oscar]

Stack utiliza lógica LIFO: último en entrar, primero en salir.

- Se agregan Antonio, Carmen, Mariana y Dante.
- El primer pop() elimina Dante, que era el último elemento ingresado.
- El segundo pop() elimina Mariana.
- Luego se agrega Oscar al final de la pila.
- La pila final queda [Antonio, Carmen, Oscar].

```
push Antonio  
push Carmen  
push Mariana  
push Dante
```

```
pop → elimina Dante  
pop → elimina Mariana  
push Oscar
```

Resultado: [Antonio, Carmen, Oscar]

Arreglo de objetos y sobrescritura de posiciones

¿Qué se imprime en consola?

```
public class MainAuto {  
    public static void main(String[] args) {  
        Auto autos[] = new Auto[3];  
        autos[0] = new Auto("Fiat", "Rojo");  
        autos[1] = new Auto("VW", "Verde");  
        autos[2] = new Auto("Gold", "Azul");  
        autos[0] = new Auto("Ford", "Negro");  
        autos[2] = new Auto("Suzuki", "Gris");  
        for (Auto a : autos) {  
            System.out.print(a.getMarca()+" ");  
        }  
    }  
}
```

a. Ford VW Suzuki

b. Este programa no se puede ejecutar, tiene un error de sintaxis.

c. Fiat VW Gol Ford Suzuki

d. Suzuki Ford Gol VW Fiat

Respuesta correcta y explicación

Correcta: a. Ford VW Suzuki

Las asignaciones posteriores reemplazan objetos anteriores del arreglo.

- El arreglo tiene 3 posiciones: 0, 1 y 2.
- Primero se cargan Fiat, VW y Gold.
- Luego `autos[0]` se reemplaza por Ford.
- Luego `autos[2]` se reemplaza por Suzuki.
- El `for-each` recorre el estado final del arreglo: Ford, VW y Suzuki.

```
autos[0] = Fiat  
autos[1] = VW  
autos[2] = Gold
```

```
autos[0] = Ford  
autos[2] = Suzuki
```

```
Estado final:  
[Ford, VW, Suzuki]
```

```
Salida: Ford VW Suzuki
```

Ordenamiento con error en el intercambio

¿Qué se imprime por consola al ejecutar este código?

```
public class Ordenamiento {  
    public static void main(String[] args) {  
        int [] numeros = {2,35,10,8};  
        miOrdenamiento(numeros);  
        System.out.println(Arrays.toString(numeros));  
    }  
  
    public static void miOrdenamiento(int[] arreglo) {  
        int n = arreglo.length;  
        for (int i = 0; i < n - 1; i++) {  
            for (int j = 0; j < n - i - 1; j++) {  
                if (arreglo[j] > arreglo[j + 1]) {  
                    arreglo[j] = arreglo[j + 1];  
                    arreglo[j + 1] = arreglo[j];  
                }  
            }  
        }  
    }  
}
```

a. [2, 8, 8, 8]

b. [35, 10, 8, 2]

c. [2, 10, 8, 35]

d. [2, 8, 10, 35]

Respuesta correcta y explicación

Correcta: a. [2, 8, 8, 8]

El algoritmo intenta ordenar, pero el intercambio está mal implementado.

- Para intercambiar dos valores se necesita una variable auxiliar.
- Aquí primero se asigna `arreglo[j] = arreglo[j + 1]`.
- Después `arreglo[j + 1]` recibe el valor ya modificado de `arreglo[j]`.
- Por eso se pierde el valor original y se duplican valores.
- El resultado final no es un ordenamiento correcto: [2, 8, 8, 8].

Inicio: [2, 35, 10, 8]

35 > 10:
[2, 10, 10, 8]

10 > 8:
[2, 10, 8, 8]

10 > 8:
[2, 8, 8, 8]

Salida: [2, 8, 8, 8]

Excepciones, catch y finally

¿Qué se imprime en la consola al ejecutar este programa?

```
public class PruebaExcepcion {
    public static void main(String[] args) {
        try {
            System.out.println(calcular());
        } catch (Exception e) {
            System.out.println("Excepcion en calcular");
        }
    }

    private static int calcular() {
        int valor = 10;
        try {
            valor++;
            valor += 10;
            valor += Integer.parseInt("e1 5");
            valor++;
        } catch (NumberFormatException e) {
            valor += Integer.parseInt("20");
        } finally {
            valor -= 5;
        }
        valor--;
        return valor;
    }
}
```

a. 40

b. 38

c. 35

d. 36

Respuesta correcta y explicación

Correcta: c. 35

La excepción se captura dentro de `calcular()` y luego se ejecuta `finally`.

- `valor` inicia en 10, luego pasa a 11 y después a 21.
- `Integer.parseInt("el 5")` produce `NumberFormatException`.
- El `catch` suma 20, por lo tanto `valor` pasa a 41.
- `finally` resta 5 y luego, fuera del `try-catch`, `valor--` deja el resultado en 35.

```
valor = 10  
valor++      → 11  
valor += 10  → 21  
parseInt("el 5") → excepción  
catch: +20   → 41  
finally: -5  → 36  
valor--      → 35
```

Interfaces y métodos default

Dadas las afirmaciones, ¿cuáles son verdaderas?

```
public interface Vehiculo {  
    void acelerar();  
    void frenar();  
  
    default void describir() {  
        System.out.println("Yo soy un vehículo.");  
    }  
}
```

- 1 - Esta interfaz está definida correctamente.
- 2 - Se puede crear una instancia de Vehiculo.
- 3 - No está definida correctamente porque no puede tener un método implementado.
- 4 - Para implementarla desde una clase se usa implements.
- 5 - Para implementarla desde una clase se usa extends.
- 6 - Si implementamos esta interfaz debemos implementar todos sus métodos abstractos.

a. 2, 3, 5, 6

b. 2, 3, 5

c. 4, 6

d. 1, 4, 6

Respuesta correcta y explicación

Correcta: d. 1, 4, 6

La interfaz es válida y se implementa con implements.

- Una interfaz puede declarar métodos abstractos.
- También puede tener métodos default con implementación.
- No se puede instanciar directamente una interfaz con new.
- Una clase que implementa la interfaz debe implementar los métodos abstractos, salvo que sea abstracta.

```
interface Vehiculo { ... }  
class Auto implements Vehiculo { ... }  
  
acelerar() y frenar()  
→ deben implementarse
```

Sobrecarga de métodos

¿Qué concepto en Java permite definir múltiples métodos con el mismo nombre pero diferentes tipos o número de parámetros?

a. Polimorfismo

b. Sobrecarga

c. Herencia

d. Sobreescritura

Respuesta correcta y explicación

Correcta: b. Sobrecarga

La sobrecarga permite reutilizar el nombre de un método cambiando su firma.

- La firma del método incluye el nombre y la lista de parámetros.
- Puede variar la cantidad, el tipo o el orden de los parámetros.
- No alcanza con cambiar solamente el tipo de retorno.
- La sobreescritura ocurre en herencia, cuando una subclase redefine un método heredado.

```
sumar(int, int)
sumar(double, double)
sumar(int, int, int)
```

Queue: add, poll y remove

¿Qué se muestra en la consola al ejecutar este programa?

```
public class Cola {  
    public static void main(String[] args) {  
        Queue<String> cola = new LinkedList<>();  
  
        cola.add("Uno");  
        cola.add("Dos");  
        cola.add("Tres");  
        cola.poll();  
        cola.remove();  
  
        System.out.println(cola);  
    }  
}
```

a. [Tres]

b. [Uno, Dos]

c. [Dos, Tres]

d. [Uno]

Respuesta correcta y explicación

Correcta: a. [Tres]

La cola trabaja con lógica FIFO: primero en entrar, primero en salir.

- Se agregan Uno, Dos y Tres en ese orden.
- poll() elimina el primer elemento de la cola: Uno.
- remove() vuelve a eliminar el primer elemento actual: Dos.
- Luego queda solamente Tres en la cola.

```
[Uno, Dos, Tres]
poll()    → elimina Uno
[Dos, Tres]
remove()  → elimina Dos
[Tres]
```

Polimorfismo y sobrescritura

¿Qué se muestra en la consola al ejecutar este programa?

```
class Persona {  
    public int caminar() { return 2; }  
}  
class Estudiante extends Persona {  
    @Override public int caminar() { return 4; }  
}  
class Deportista extends Persona {  
    @Override public int caminar() { return 8; }  
}  
class PersonaMayor extends Persona {  
    @Override public int caminar() { return 1; }  
}  
  
public class Principal {  
    public static void main(String[] args) {  
        Persona[] personas = new Persona[4];  
        personas[0] = new Persona();  
        personas[1] = new Estudiante();  
        personas[2] = new Deportista();  
        personas[3] = new PersonaMayor();  
  
        int suma = 0;  
        for (int i = 0; i < personas.length; i++) {  
            suma += personas[i].caminar();  
        }  
        System.out.println(suma);  
    }  
}
```

a. 15

b. 8

c. 20

d. 10

Respuesta correcta y explicación

Correcta: a. 15

Java invoca el método según el objeto real almacenado en cada posición.

- El arreglo es de tipo Persona, pero contiene objetos de subclases.
- Cada subclase sobrescribe el método caminar().
- En tiempo de ejecución se usa la versión correspondiente al objeto real.
- La suma resultante es $2 + 4 + 8 + 1 = 15$.

```
Persona    → 2
Estudiante → 4
Deportista → 8
PersonaMayor → 1
Total      → 15
```

ArrayList: add, set y remove

¿Qué se muestra en la consola al ejecutar este programa?

```
public class EjemploListaFrutas {  
    public static void main(String[] args) {  
        List<String> frutas = new ArrayList<>();  
        frutas.add("Manzana");  
        frutas.add("Banana");  
        frutas.add("Pera");  
        frutas.add(1, "Durazno");  
        frutas.set(3, "Naranja");  
        frutas.add("Kiwi");  
        frutas.add(2, "Uva");  
        frutas.remove("Banana");  
        frutas.remove("Banana");  
        frutas.remove(0);  
        frutas.remove(2);  
        System.out.println(frutas);  
    }  
}
```

a. [Durazno, Uva, Kiwi]

b. [Durazno, Uva, Naranja, Kiwi]

c. Se produce un error al ejecutar este programa.

d. [Durazno, Naranja, Kiwi, Pera]

Respuesta correcta y explicación

Correcta: a. [Durazno, Uva, Kiwi]

Las operaciones modifican posiciones y eliminan elementos de la lista.

- `add(1, "Durazno")` inserta y desplaza los elementos siguientes.
- `set(3, "Naranja")` reemplaza el elemento en la posición 3.
- `remove("Banana")` elimina la primera coincidencia; la segunda llamada no cambia la lista.
- `remove(0)` y `remove(2)` eliminan por índice; el resultado final es `[Durazno, Uva, Kiwi]`.

```
[Manzana, Banana, Pera]
→ [Manzana, Durazno, Banana, Pera]
→ [Manzana, Durazno, Uva, Banana, Naranja, Kiwi]
→ [Durazno, Uva, Kiwi]
```

Síntesis para repasar

Conceptos clave trabajados

- Excepciones, NumberFormatException, catch y finally.
- Interfaces, métodos default e implementación con implements.
- Sobrecarga de métodos y diferencia con sobrescritura.
- Estructuras de datos: Queue, Stack, ArrayList y arreglos.
- Polimorfismo: ejecución del método según el objeto real.
- Trazado paso a paso de variables, condiciones y salida por consola.

Recomendación para el examen del 17/07: antes de elegir una opción, realizar una traza manual de variables, estructuras y salida esperada.